

Functional Verification of AMBA AHB-Lite using Layered Testbench Technology of System Verilog

Ashima Gandhi*, Neeraj Kr. Shukla

VLSI Group, Department of Electronics and Electrical Communications Engineering,
The Northcap University, Gurgaon, Haryana, India

Abstract

Huge complexity of chip increased in the past recent years and integration of more number of transistors on a single system on chip (SoC) makes verification of any SoC based design very critical. Ever increasing complexity and size of designs lead to a number of verification challenges, which are the productivity, efficiency, reusability and completeness. About 75–80 % of the total design cycle time is spent in functional verification. To reduce this time and effort, hardware verification languages came into picture. The most popularly used HVL is system Verilog which is based on object oriented programming concepts (OOPs). A design can be verified by writing a verification written in system verilog. Basically, shortening-voltage (SV) can be used for both design and verification, so it is termed as hardware design and verification language (HDVL). The work embodied in this paper presents the verification of advanced high performance bus-lite (AHB-Lite), which is the subset of advanced microcontroller bus architecture (AMBA). The verification environment is created using system verilog; and functional coverage is computed to ensure functional correctness of the design. The functional coverage was found to be 100%.

Keywords: Advanced microcontroller bus architecture, advanced high performance bus-lite, system verilog, system on chip, functional coverage, verification, hardware design and verification language, design under verification

***Author for Correspondence** E-mail: ashima.gandhi0212@gmail.com

INTRODUCTION

The structure in this paper includes the following:

- Introduction (need of verification, different functional verification approaches and hardware design and verification language, system verilog),
- Explanation of device under test (DUT) taken as architecture, block diagram, operation and list of signals, etc.,
- The test-bench architecture and its components,
- The simulation results
- Conclusion.

Need of Verification

Ever increasing complexity and size of designs lead to a number of verification challenges, which are productivity, efficiency, reusability and completeness. Productivity includes two things, i.e., one is the ability to handle complex designs in less amount of time and the other is to move to high levels of

abstraction in terms of verification utilities and functional blocks that are being verified. Efficiency gives the measure of human effort and time required to complete a verification task. It can be increased by automation and tool utilities. Reusability refers to the ability of reusing an existing environment for new projects. Completeness means covering as much of the design functionality to the maximum extent possible.

In the process of chip manufacturing and testing, verification is the most integral part and is as important as the designing. The motive behind verification is to check if the design developed is as per the specifications or not. It is recognized as the largest task in silicon development and as such has the great impact on several factors like TTM (time to market), cost and the most important is quality. During development of chips, it is essential that the functionality embedded in it, should be verified thoroughly because it is

very hard or impossible to fix them once they have been manufactured. According to the statistics, 75–80% of the total design time is spent in doing the verification. Hardware verification languages were the need of hour to reduce this time and effort.

Basically, the verification approach is based on four fundamental concepts as:

- i. Raising the level of abstraction of verification tasks
- ii. Constrained random generation of verification data and scenarios
- iii. Automatic checking of simulation results
- iv. Coverage collection to measure verification progress and guide random generation to cover missing scenarios.

Verification is not a one-step process. It consists of five main phases, namely; verification plan development, verification infrastructure implementation, verification environment bring-up, constrained random verification, corner case verification.

Different Functional Verification Approaches

The aim of the verification is to verify that the logic design fulfills the requirements in the specifications. Does the design operate as per the specifications? The three functional verification approaches are black box verification, white box verification and grey box verification.

- i. **Black Box Verification:** The verification engineer is unaware about the structure and implementation of the actual design. The verification is performed through the available interfaces without any access to the internal design code. Hence, this approach results in lack of visibility, availability and controllability. The delays between the occurrence of a problem and appearance of its symptoms at the outputs make it difficult to recognize the response from the input and find out the cause of problem. The test-bench is portable in this case. The advantage of the black box verification is that it is not implementation specific i.e. the verification engineer need not worry whether the design is implemented in RTL code or any other software.

- ii. **White Box Verification:** The verification engineer has full controllability and observability of the internal structures in this approach i.e. how the design is structured and implemented. So, it becomes easy to access the design code and detect the bugs and their cause of occurrence quickly. The white box approach has the drawback that it is implementation specific. So, the testbench generated for the design cannot be used for the RTL code design i.e. the testbench is not portable.

- iii. **Grey Box Verification:** The grey box design approach combines the feature of both the black box approach and white box approach. The design is observed and controlled through the top level interfaces. But the verification engineer can access specific features of the design. For SoC design and system-level verification, the system is assumed as a design composed of different blocks where each block is treated as a black box and the system as a whole is treated as a white box.

Hardware Design and Verification Language: System Verilog

A hardware verification language or HVL is a programming language, which is used in verifying the RTL design written in a HDL (Hardware Description Language) such as Verilog. HVLs basically incorporates features of a high-level programming language like C++/Java plus the features for easy bit-level manipulation similar to those found in HDLs. HVLs provide constrained random stimulus generation and functional coverage constructs; which helps in verification of complex hardware. Some commonly used HVLs are: System Verilog, OpenVera, e, and SystemC. System Verilog, which is mostly used in the industry attempts to combine HDL and HVL constructs into a single standard; that's why it is sometimes called as HDVL.

The main feature of SV includes:

- i. Constraint Randomization.
- ii. Easy to integrate with C models.
- iii. OOPs support.
- iv. Introduction of new data types i.e. logic and wire.
- v. Assertions.
- vi. Dynamic memory allocation.

- vii. Allows the use of two state data type, which makes the simulator more efficient.
- viii. Provides coverage support.

Although, it has various important advantages, there are two drawbacks also: one is limited synthesis tool support and the other is it doesn't support parallelism. Also, it supports reusability to a very less extent. SV has a layered test bench architecture; which is divided into four layers, i.e., signal layer (bottom most), command layer, functional layer and scenario layer (top most).

Figure 1 shows the layered test-bench architecture and the components at each layer.

- i. **Signal Layer:** The signal layer forms the bottommost layer of the layered testbench architecture. It includes the design under test (DUT) and the signals that connect it to the testbench.
- ii. **Command Layer:** This layer lies above the signal layer. The command layer consists of the driver and monitor as the main blocks. The driver takes the transaction from the agent and breaks down into the individual signal and then drive those signals to the DUT whereas the monitor performs the opposite task. It takes the signals from the DUT and then groups it into the transaction and sends them to the checker.

iii. **Functional Layer:** This layer lies above the command layer. Agent, scoreboard and checker blocks are present in this layer. The agent takes the higher level transactions and breaks down into individual transactions such as read and writes commands. The same command is sent to the scoreboard. The checker block compares the contents from the monitor with the contents of the scoreboard.

iv. **Scenario Layer:** This layer is the next topmost layer in the testbench hierarchy. It contains the generator that arranges the different scenarios. The generator receives the different user defined test.

DUT- AMBA AHB-Lite

In system on chip (SoC) design, AMBA acts as on chip bus. Earlier it found application in the microcontroller devices but with the advance of time it is used in many application specific integrated circuits (ASICs) parts and system on chip design chips parts [2]. The purpose of the advanced microcontroller bus architecture is to connect and manage the functional blocks present in the system on chip designs. It enables the design of the multiprocessor, which contains many peripherals and controllers (Figure 2). AMBA bus architecture consists of three components as, advanced high performance bus (AHB), advanced system bus (ASB) and advanced peripheral bus (APB) respectively [3].

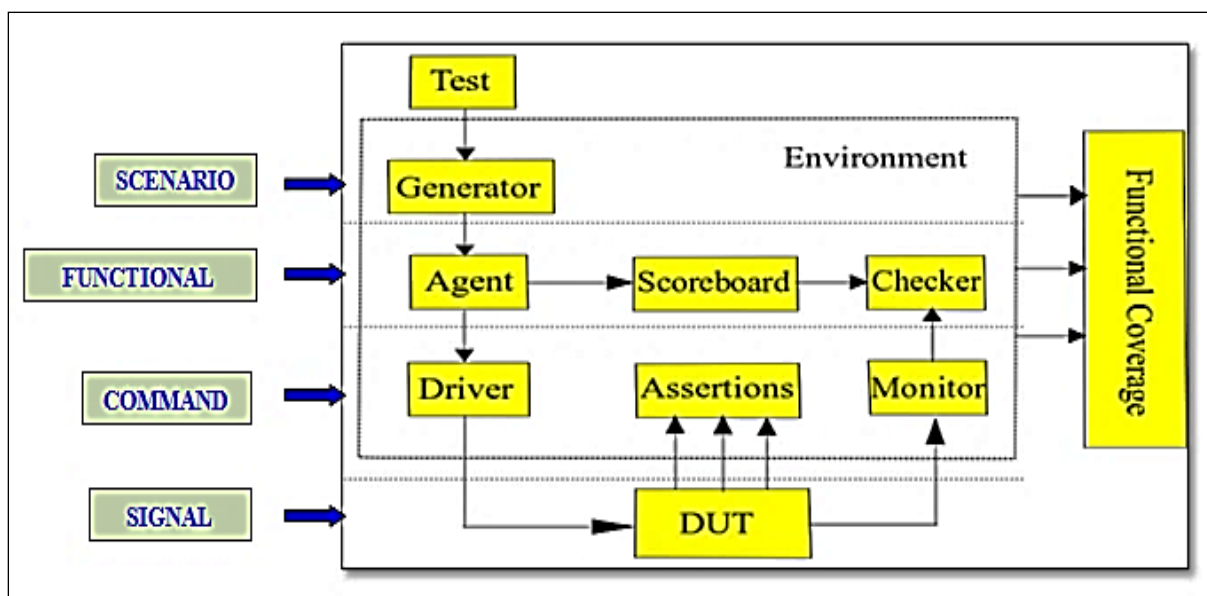


Fig. 1: Layered Test-bench Architecture [1].

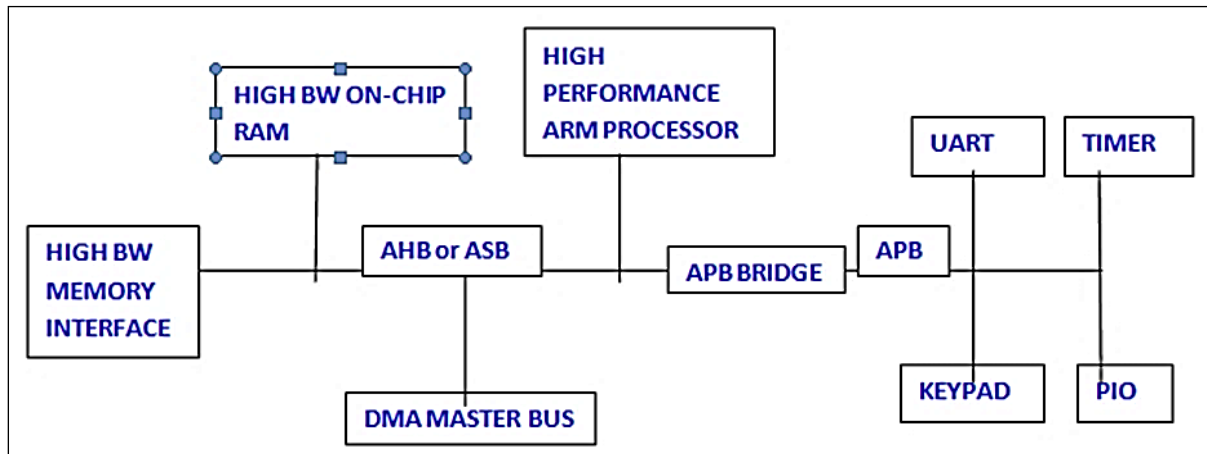


Fig. 2: AMBA Bus Architecture [3].

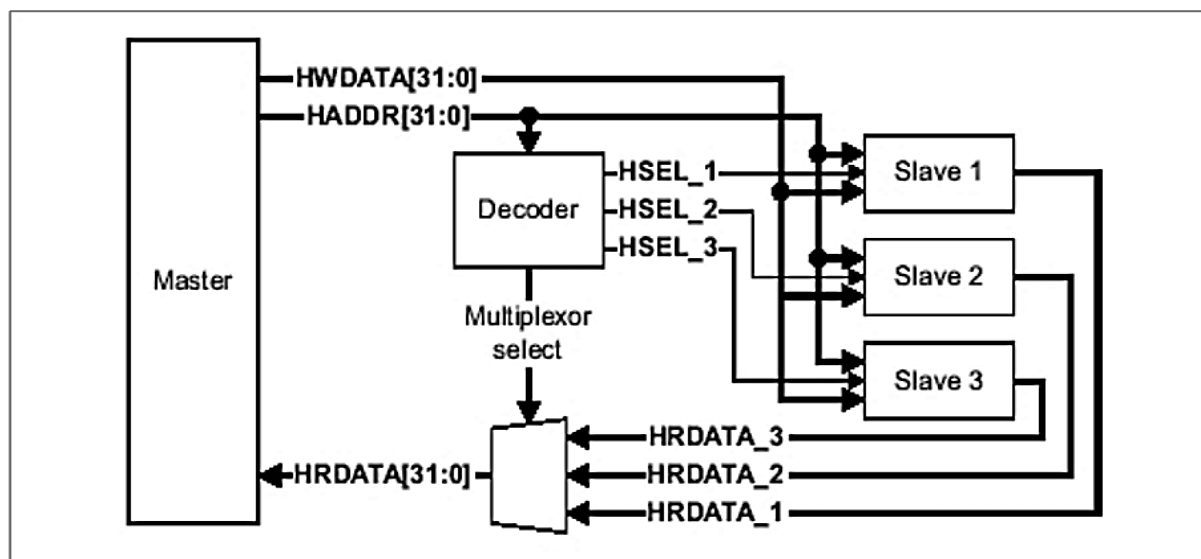


Fig. 3: AHB-Lite Block Diagram [3].

HB-Lite is the subset of AHB protocol and was introduced in AMBA 3.0 specification. AHB is a communication bus protocol, designed by ARM Ltd., in their AMBA 2.0 specification [3]. However, both are communication bus protocols, but there are two main differences between the two; AHB-Lite is a single master protocol and no arbiter is required in AHB-Lite. AMBA AHB-Lite protocol addresses the requirements of high-performance synthesizable designs. It supports a single master and provides high-bandwidth operation. AHB-Lite implements the features required for high-performance, high clock frequency systems including [3]:

- Burst transfers,
 - Single-clock edge operation,
 - Non-tristate implementation,
- Wide data bus configurations, 64, 128, 256, 512, and 1024 bits.

Figure 3 shows AHB-Lite system design with one master and three slaves. The bus interconnects logic between the master and slave consists of one address decoder and one slave-to-master multiplexor. So, the components of AHB-Lite system are master, slave, decoder and multiplexor.

Operation of AMBA AHB-Lite

The data transfer is initiated by the master by driving the address and control signals. The information about the address, direction, width of the transfer, and indicate if the transfer forms part of a burst; is given by these signals. Every transfer consists of two phases namely: address phase and data phase. Address phase contains one address and control cycle whereas data phase can contain one or more data cycles [4–6]. The transfer type can be busy, idle, non-seq or seq, depending upon the

value of HTRANS (a two bit signal from master to slave). The write data bus i.e. HWDATA moves data from master to slave whereas read data bus, i.e. HRDATA moves data from slave to master. To indicate the success or failure of transfer, slave sends an acknowledgement signal known as HRESP, whose values can be transfer done, transfer pending and error response.

List of AHB-Lite Signals

- a. HCLK: The clock is an input to all elements in an AHB system and is assumed to come from some external clock generator.
- b. HRESETn: This active low signal is an input to all elements in AHB.
- c. HADDR: A 32 bit bus, output from the master, which indicates the address to be used for a transfer.
- d. HSIZE: This signal is master's output, which indicates the size of the read or writes transfer to be performed at that time.
- e. HTRANS: An output from the master (input to slave) which shows the type of transfer to be done.
- f. HWRITE: An output from the master, which shows the direction of data transfer.
- g. HBURST: The master generates this bus to tell the system about the kind of burst it is performing.
- h. HPROT: This allows the master to tell the slave characteristics of the transfer.
- i. HWDATA: The master output which contains the data it wants to write to an address in a slave.
- j. HRDATA: The slave output (input to the master) which contains data that is read from a slave.
- k. HSELx: A set of system-specific decoder outputs, generated combinatorial from the decoder's HADDR input.
- l. HRESP: An output from the slave (input to the master) which indicates whether the transfer was successful.
- m. HREADY: This is an output from each slave which shows when the transfer is completed.
- n. HLOCK/HMASTLOCK: Indicates a locked transfer.

Read and Write Operations in AHB-Lite Protocol

In read transfer, HWRITE must be low and the slave must generate the data on the read data bus i.e. on the data line HRDATA [31:0]. Similarly, in write transfer; HWRITE must be high and the master must write the data on the write data bus, i.e. on the data line HWDATA [31:0]. Read transfers and write transfers are depicted in Figures 4 and 5 respectively [6].

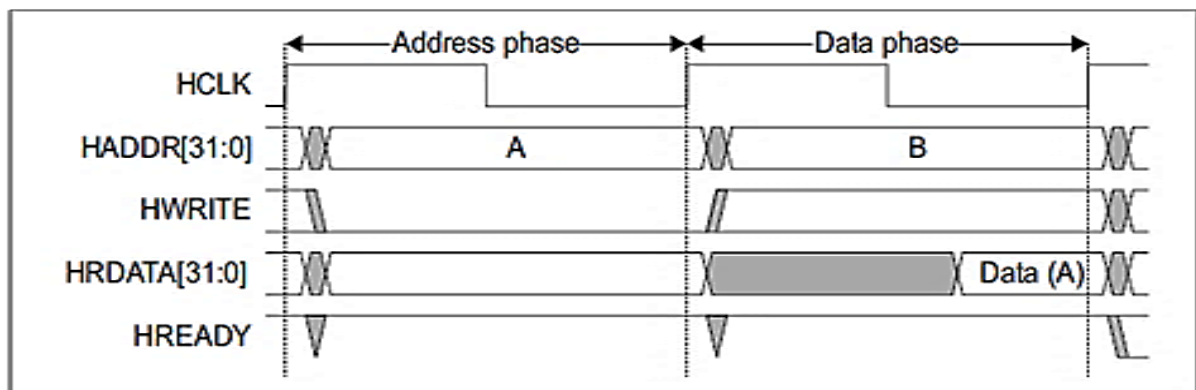


Fig. 4: Read Transfer in AHB-Lite [3].

These are simple transfers with no wait states. After the rising edge of HCLK, the master drives the address and control signals on the bus. The slave then samples the address and control information on the next arriving positive edge of HCLK [5]. After the address

and control signal being sampled by the slave, it can start to drive the appropriate HREADY (values: 0 and 1 which indicates extended transfer and transfer complete respectively) response. This response is sampled by the master on the third rising edge of HCLK.

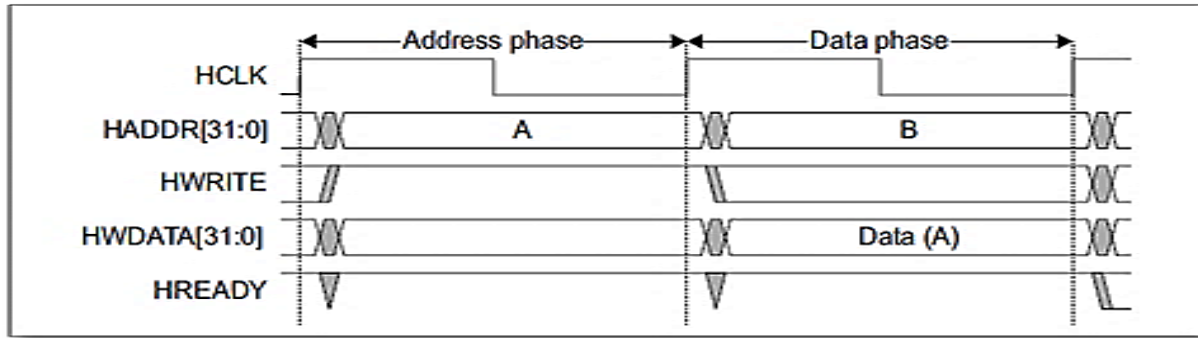


Fig. 5: Write Transfer in AHB-Lite [3].

TESTBENCH ARCHITECTURE OF SYSTEM VERILOG VERIFICATION ENVIRONMENT

The various building blocks in SV environment shown in Figure 6 are described as follows:

- i. **Interface:** The role of an interface is to encapsulate the interconnection and communication between the blocks. In case, a new port is to be added to the existing ones, there is no need to make changes in the blocks, the new signal is just added to the interface. There are several advantages of using interface instead of defining the signals individually such as enhances the reusability, easy to compile in an independent file, easily maintained since adding and removing the signal is simple, the probability of errors is considerably reduced. The interface is instantiated like a module but is connected to ports like a signal. All input and output signals are declared in the interface including clock signal. The use of the clocking block adds synchronization to the given clock. Two clocking blocks as well as two modports are defined, one each for monitor and driver.
- ii. **Transactor:** The role of transactor is to randomize the variables used in our DUT. Also, constraints can be applied in transactor so as to make our environment to cover legal combinations for variables. Transactor performs the high level operations.
- iii. **Generator:** The role of generator is to generate stimulus which is sent to the design under test by driver. The constraints that are defined in stimulus are combinational in nature but those defined in stimulus generator are sequential in nature. The output from generator flows

into driver as well as scoreboard. Covergroup is defined in this block so as to analyze functional coverage. The randomize function is also called in generator which assigns new values to random variables and makes sure that all constraints are satisfied. Threads are nothing but commands. Mailboxes are used to transfer information from generator thread to driver thread. Mailbox is just a FIFO with a source and a sink. Another mailbox is used to transfer information from generator to scoreboard.

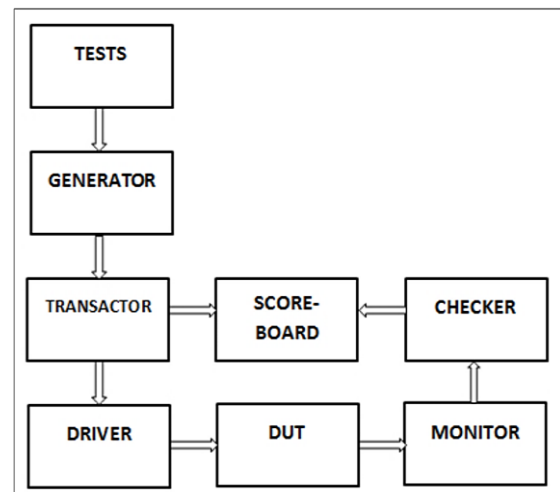


Fig. 6: Building Blocks of SV Verification Environment.

- iv. **Driver:** Driver is the block that translates the operations produced by generator into actual inputs for design under verification. Generator creates inputs at high level of abstraction, as transactions like read, write operations. Driver converts these inputs into actual design inputs, as they are defined in the specifications of design interface. A single mailbox is used from generator to driver.

- v. **Monitor:** The role of monitor is to drive output signals of DUT. It converts state of design and its outputs to a transaction abstraction level so that it can be stored in scoreboards and checked later on i.e. it converts pin level activity into high level. Since its output flows into scoreboard, a mailbox from monitor to scoreboard is used.
- vi. **Scoreboard:** Role of scoreboard is to store expected DUT output. The stimuli generated by generator and sent to driver, are also stored in scoreboard, until output comes out of DUT. After the output comes out of DUT, scoreboard checks whether the inputs from generator matches the output from monitor. It uses two mailboxes, from generator to scoreboard and monitor to scoreboard.
- vii. **Environment:** The instances of all the verification components are included in the environment.
- viii. **Program:** The program block serves a separator between design and testbench. It specifies specialized execution semantics in reactive region for all elements declared within the program. It provides race-free interaction between DUT and testbench. A module in Verilog describes hardware; it

can contain always, initial and assign statements. To have a clear separation between testbench and design, and getting the environment properly initialized and synchronized, avoiding races between design and testbench, a program block is used.

- ix. **Top:** The role of top block is to generate clock signal and reset signal. In top block, three blocks are called i.e. Interface, DUT and Program, which covers the whole environment for Verification.

SIMULATION RESULTS

For computing the functional coverage of AHB-Lite protocol, covergroups and coverpoints are defined in generator.sv file. For all the input signals, coverpoints are defined; which are included in the covergroup, i.e. cover_ahb as shown in the above snapshot (Figure 7). According to the protocol specifications, the values of the signals which are not required to be checked are taken in ignore_bins. For example, for input signals SIZE and BURST, concept of ignore_bins is used [7]. To limit the number of bins, option.auto_bin_max is used; generally the number of bins created for every coverpoint is 64.

```
covergroup cover_ahb;

    HREADY: coverpoint w.HREADY;
    HRESP: coverpoint w.HRESP;
    WRITE: coverpoint w.WRITE;
    SIZE: coverpoint w.SIZE {ignore_bins hi = {6,7}};
    BURST: coverpoint w.BURST {ignore_bins ig = {5,6,7}};
    HRDATA: coverpoint w.HRDATA iff (!w.HRESETn) { option.auto_bin_max
= 10; }
    DATA: coverpoint w.DATA iff (!w.HRESETn) { option.auto_bin_max = 10;
}
    ADDR: coverpoint w.ADDR iff (!w.HRESETn) { option.auto_bin_max = 10;
}
endgroup
```

Fig. 7: Covergroup and Coverpoints of AHB-Lite.

COVERGROUP COVERAGE:			
Covergroup	Metric	Goal/ Status At Least	
TYPE /program_sv_unit/generator/cover_ahb	100.0%	100	Covered
Coverpoint cover_ahb::HREADY	100.0%	100	Covered
Coverpoint cover_ahb::HRESP	100.0%	100	Covered
Coverpoint cover_ahb::WRITE	100.0%	100	Covered
Coverpoint cover_ahb::SIZE	100.0%	100	Covered
Coverpoint cover_ahb::BURST	100.0%	100	Covered
Coverpoint cover_ahb::HRDATA	100.0%	100	Covered
Coverpoint cover_ahb::DATA	100.0%	100	Covered
Coverpoint cover_ahb::ADDR	100.0%	100	Covered
CLASS generator			
Covergroup instance \program_sv_unit::generator::cover_ahb	100.0%	100	Covered
Coverpoint HREADY	100.0%	100	Covered
covered/total bins:	2	2	
missing/total bins:	0	2	
bin auto[0]	1016	1	Covered
bin auto[1]	984	1	Covered
Coverpoint HRESP	100.0%	100	Covered
covered/total bins:	2	2	
missing/total bins:	0	2	
bin auto[0]	993	1	Covered
bin auto[1]	1007	1	Covered
Coverpoint WRITE	100.0%	100	Covered
covered/total bins:	2	2	
missing/total bins:	0	2	
bin auto[0]	1001	1	Covered
bin auto[1]	999	1	Covered
Coverpoint SIZE	100.0%	100	Covered
covered/total bins:	6	6	
missing/total bins:	0	6	
ignore_bin hi	500		Occurred
bin auto[0]	250	1	Covered
bin auto[1]	250	1	Covered
bin auto[2]	250	1	Covered
bin auto[3]	250	1	Covered
bin auto[4]	250	1	Covered
bin auto[4]	250	1	Covered
bin auto[5]	250	1	Covered
Coverpoint BURST	100.0%	100	Covered
covered/total bins:	5	5	
missing/total bins:	0	5	
ignore_bin ig	750		Occurred
bin auto[0]	250	1	Covered
bin auto[1]	250	1	Covered
bin auto[2]	250	1	Covered
bin auto[3]	250	1	Covered
bin auto[4]	250	1	Covered
Coverpoint HRDATA	100.0%	100	Covered
covered/total bins:	10	10	
missing/total bins:	0	10	
bin auto[0:429496728]	197	1	Covered
bin auto[429496729:858993457]	209	1	Covered
bin auto[858993458:1288490186]	204	1	Covered
bin auto[1288490187:1717986915]	199	1	Covered
bin auto[1717986916:2147483644]	192	1	Covered
bin auto[2147483645:2576980373]	193	1	Covered
bin auto[2576980374:3006477102]	200	1	Covered
bin auto[3006477103:3435973831]	202	1	Covered
bin auto[3435973832:3865470560]	206	1	Covered
bin auto[3865470561:4294967295]	198	1	Covered
Coverpoint DATA	100.0%	100	Covered
covered/total bins:	10	10	
missing/total bins:	0	10	
bin auto[0:429496728]	202	1	Covered
bin auto[429496729:858993457]	200	1	Covered
bin auto[858993458:1288490186]	199	1	Covered
bin auto[1288490187:1717986915]	199	1	Covered
bin auto[1717986916:2147483644]	200	1	Covered
bin auto[2147483645:2576980373]	204	1	Covered
bin auto[2576980374:3006477102]	199	1	Covered
bin auto[3006477103:3435973831]	200	1	Covered
bin auto[3435973832:3865470560]	198	1	Covered
bin auto[3865470561:4294967295]	199	1	Covered
Coverpoint ADDR	100.0%	100	Covered
covered/total bins:	10	10	
missing/total bins:	0	10	
bin auto[0:429496728]	199	1	Covered

Coverpoint ADDR	100.0%	100 Covered
covered/total bins:	10	10
missing/total bins:	0	10
bin auto[0:429496728]	199	1 Covered
bin auto[429496729:858993457]	200	1 Covered
bin auto[858993458:1288490186]	200	1 Covered
bin auto[1288490187:1717986915]	199	1 Covered
bin auto[1717986916:2147483644]	202	1 Covered
bin auto[2147483645:2576980373]	195	1 Covered
bin auto[2576980374:3006477102]	203	1 Covered
bin auto[3006477103:3435973831]	202	1 Covered
bin auto[3435973832:3865470560]	200	1 Covered
bin auto[3865470561:4294967295]	200	1 Covered
TOTAL COVERGROUP COVERAGE: 100.0% COVERGROUP TYPES: 1		

Fig. 8: Coverage Report Analysis of AHB-Lite Protocol: F. Coverage= 100%

Figure 8 shows functional coverage of AHB-Lite protocol using SV verification environment; it's found out to be 100%, which shows the design is functionally correct. The cover groups and coverpoints are written in generator.sv file. All the coverpoints namely HREADY, HRESP, WRITE, SIZE, BURST, HRDATA, DATA & ADDR of covergroup cover_ahb are covered completely.

CONCLUSION

This paper gives an overview of AMBA bus architecture and elaborates AHB-Lite protocol. The paper has illustrated how a SV environment can be used to verify a RTL design like an AHB-Lite protocol. The environment is created in SV completely wraps around the DUT. Functional coverage is computed for AHB-Lite to ensure correctness of the design. The functional coverage is found to be 100%.

ACKNOWLEDGEMENT

The authors are grateful to their organization for its help, support and guidance.

REFERENCES

1. Chris S. System Verilog for Design: A Guide to Using System Verilog for Hardware Design and Modeling. Springer Second Edn, 2008.

2. Tejaswini HN, Revati B, Ravishankar CV. Verification of AHB Protocol using UVM. *International Journal of Engineering Research & Technology*. June 2014; 3(6):
3. ARM, AMBA Specification Overview. <http://www.arm.com>.
4. Vanessa RC. Getting Started with UVM: A Beginner's Guide. Verilab. 2013.
5. Richa S, Akhilesh K, Archana KS. Verification Analysis of AHB-LITE Protocol with coverage. *International Journal of Advances in Engineering & Technology*. Jan 2012; 2(1): 121p.
6. http://en.wikipedia.org/wiki/Advanced_Microcontroller_Bus_Architecture.
7. Ravi Y, Prasad CY. System Verilog Methodology for Verification of AHB_LITE Protocol. *International Journal for Technological Research In Engineering*. Jun 2015; 2(10).

Cite this Article

Ashima Gandhi, Neeraj Shukla. Functional Verification of AMBA AHB-Lite using Layered Testbench Technology of System Verilog. *Journal of VLSI Design Tools and Technology*. 2016; 6(2): 104–112p.